vague-sw-p8-vague-noshared

An AgentMPI run analysis

Abstract

Eight real agents were each given one module of a small SQL engine, a specification with every module signature stripped out of it, and no shared window through which to publish or read interfaces. It is the hardest cell of a two-by-two ablation and the intended upper bound on defensive coupling. The population still reached a green build: 55 of 60 acceptance cases at first integration and 60 of 60 after one repair round, agreed unanimously by `agree` at 847.5 s. It did so by refusing to depend on anyone. Three of the eight declared dependency edges were honoured in round one and five of eight in round two; the owner of `parser.py` reimplemented both the lexer and the syntax tree rather than import modules whose interfaces it had not seen, leaving `nodes.py` — 378 lines that another agent wrote — imported by nothing, and `api.py` discovered its neighbours at run time with `getattr` and `inspect.signature`. The resulting tree is 3,556 lines against the 2,195 lines the same modules, the same specification and the same commit produced seven minutes earlier when the window was available. The most important number here is not the pass rate, which is identical in both arms, but the shape of the bill: through round one the two arms cost almost the same (82,298 tokens without the window, 86,473 with it), and the entire 2.2× difference in total cost is the review-and-repair round that only the windowless arm needed. Withholding the interface channel did not make the first attempt more expensive; it made a second attempt necessary.

1 What this run is

property	value
run	`vague-sw-p8-vague-noshared`
experiment	`software`
campaign label	`vague-sw`
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 46
events	482
wall time	847.50 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	3.81×	total busy time over wall time
parallel efficiency	47.7%	achieved parallelism per rank
idle fraction	52.3%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	24.4%	time with exactly one rank busy
load imbalance	1.88×	slowest rank's busy time over the mean
coordination share	88.3%	wall time inside collectives
rank reattachments	46	fresh agent processes that took over a rank role
agent calls	32	invocations that reached a model
agent latency p50 / p95	97.03 / 252.58 s	per invocation
messages	81	59 eager, 22 rendezvous
tokens sent	169,184	128,769 deferred under rendezvous
tokens in / out	102,613 / 88,989	prompt and completion
cost	$1.6427	0.0185 $/kTok out

3 What the analysis notices

- [warning] `neighbor_allgather/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [warning] `neighbor_allgather/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [note] Collectives account for 88.3% of wall time, so this run spent more time coordinating than working.
- [note] Load imbalance is 1.88×: the slowest rank was busy far longer than the mean.
- [ok] 46 rank reattachment(s) occurred, up to incarnation 8: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 10 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

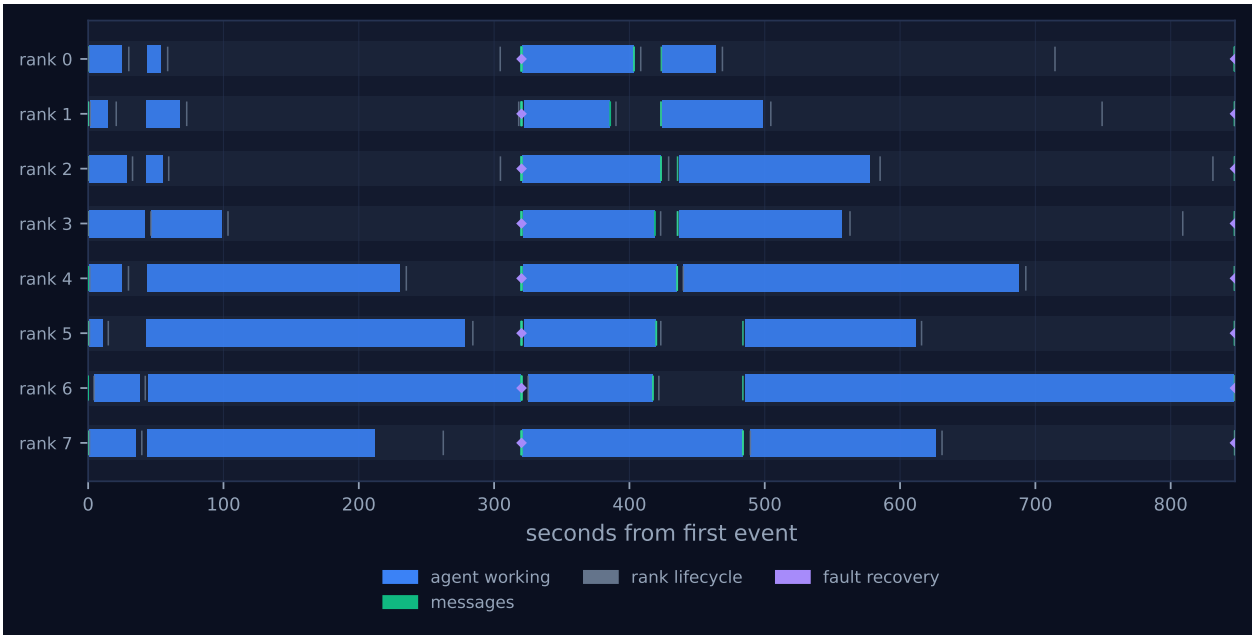

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

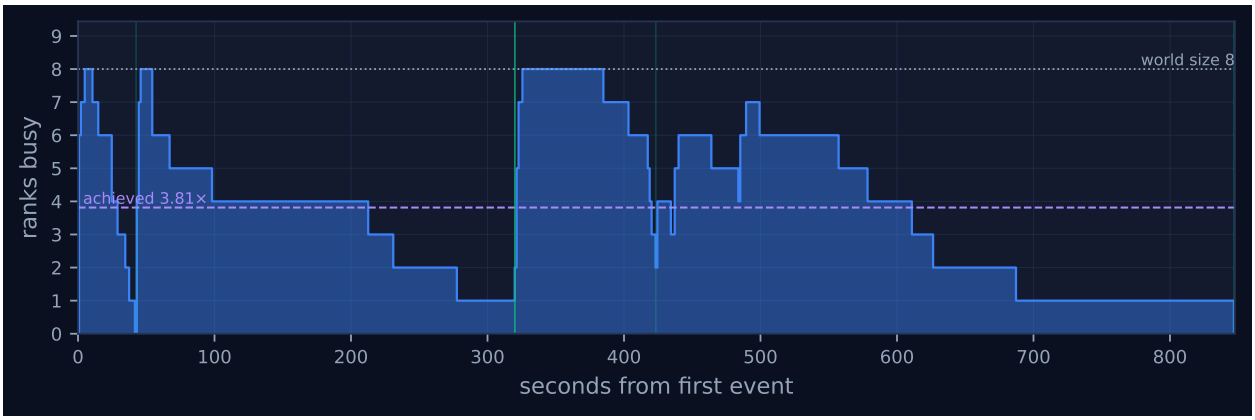

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

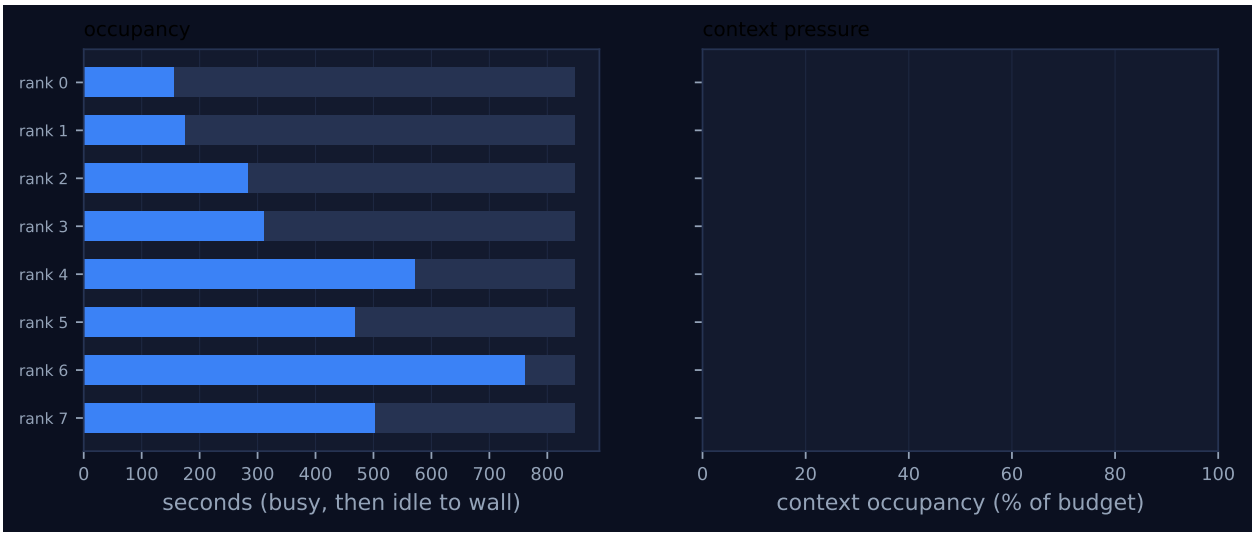

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	156.67	18.5	4	4	19	10	8,641	0.0	finalized
1	175.19	20.7	4	4	6	9	9,457	0.0	finalized
2	284.15	33.5	4	4	11	11	18,955	0.0	finalized
3	311.79	36.8	4	4	6	9	10,171	0.0	finalized
4	571.76	67.5	4	4	16	13	60,787	0.0	finalized
5	468.77	55.3	4	4	6	9	23,630	0.0	finalized
6	761.48	89.9	4	4	11	11	28,552	0.0	finalized
7	502.61	59.3	4	4	6	9	8,991	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

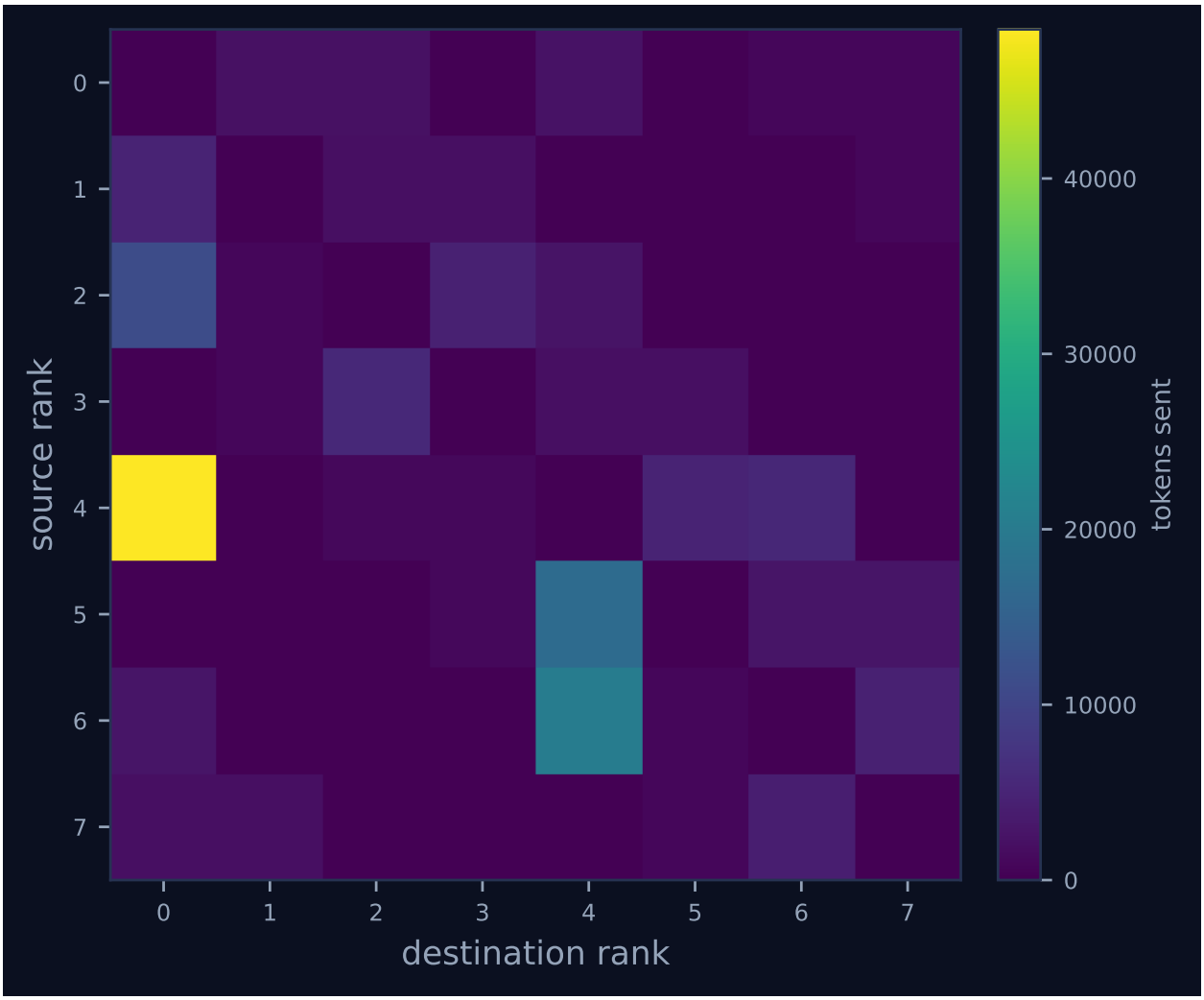

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

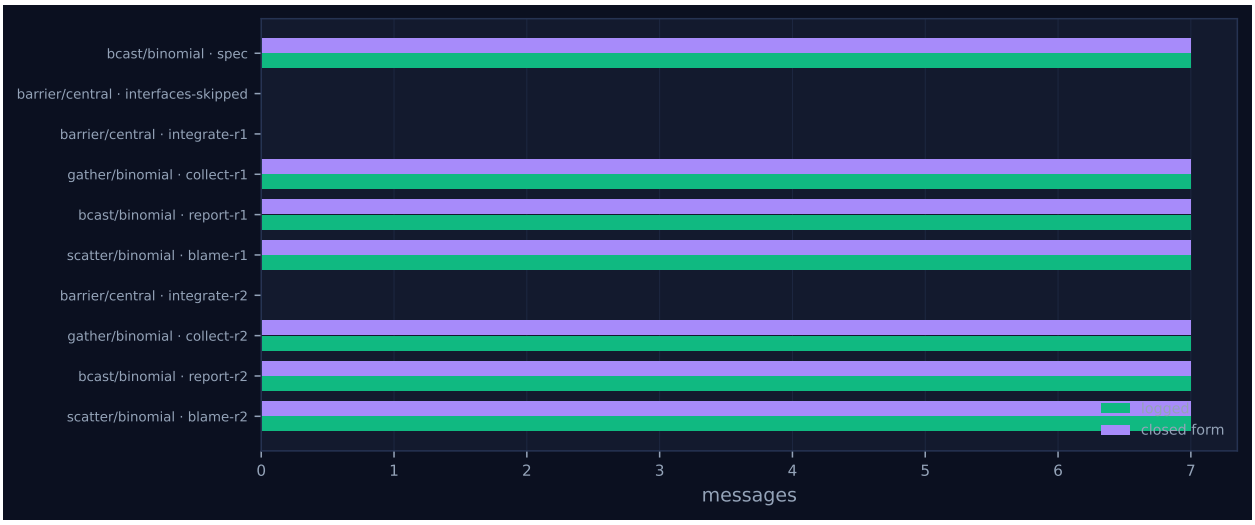

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	mo
bcast	binomial	spec	8	8	3	3	7	7	10,787	0.03	0.03	✓
barrier	central	interfaces-skipped	8	8	0	2	0	0	0	31.20	0.21	✓
barrier	central	integrate-r1	8	8	0	2	0	0	0	266.36	0.26	✓
gather	binomial	collect-r1	8	8	3	3	7	7	51,961	0.25	0.27	✓
bcast	binomial	report-r1	8	8	3	3	7	7	812	0.43	0.09	✓
scatter	binomial	blame-r1	8	8	3	3	7	7	1,164	0.00	0.09	✓
neighbor_allgather	?	review-r1	8	8	0	–	16	–	0	0.07	0.04	-
neighbor_allgather	?	reviewback-r1	8	8	0	–	16	–	0	66.53	60.55	-
barrier	central	integrate-r2	8	8	0	2	0	0	0	382.92	0.24	✓
gather	binomial	collect-r2	8	8	3	3	7	7	55,068	0.26	0.28	✓
bcast	binomial	report-r2	8	8	3	3	7	7	749	0.43	0.13	✓
scatter	binomial	blame-r2	8	8	3	3	7	7	24	0.00	0.12	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has four bar clusters per lane and they are the four agent phases of the harness, separated by the three collectives that dominate the run's wall time. From 0 to 42.5 s every rank publishes its interface. From 42.5 to 319.8 s every rank implements its module. From 320.3 to 483.9 s every rank reviews two peers. From 423.4 to 846.7 s every rank implements again. Nothing else happens: 32 agent invocations, four per rank, no retries, no contract violations and no degraded calls.

The gaps are all the same gap. Rank 0 owns `errors.py` and finished its round-one implementation 11.0 s after starting it, at $t = 53.7$; it then sat at the `integrate-r1` barrier for the 266.4 s the barrier reports as its wall, because rank 6 owns `engine.py` and its round-one agent call took 277.1 s. The same thing happens again and worse in round two: rank 0 finished at $t = 464.1$ and waited 382.9 s

for rank 6, which finished at $t = 846.7$. Those two waits are 649.3 s, or 76.6 % of the 847.5 s the run took, spent by one rank doing nothing. Rank 0's 18.5 % occupancy in the per-rank table is that fact restated.

The straggler is not random, and this is the substantive point about the shape. Round-one agent latencies, ordered, are 11.0 s for `errors`, 12.5 s for `nodes`, 25.0 s for `tokens`, 56.5 s for `functions`, 169.6 s for `api`, 188.1 s for `parser`, 235.6 s for `planner` and 277.1 s for `engine`. That is exactly the order of the module dependency graph, from its sources to its sink, and the per-rank busy times in the generated table reproduce it: 156.7, 175.2, 284.2, 311.8, 571.8, 468.8, 761.5 and 502.6 s for ranks 0 through 7, which own `errors`, `tokens`, `nodes`, `functions`, `parser`, `planner`, `engine` and `api` respectively. The 1.88× load imbalance is therefore structural rather than stochastic: the harness assigns module i to rank $i \bmod p$, the modules are of very unequal difficulty, and a barrier after every round converts that inequality directly into idle rank-seconds. A scheduler that knew the DAG could not fix this either, because there is only one `engine.py` and someone has to write it.

Figure 2 is the same story as a sawtooth. Concurrency is 8 at the start, decays to 1 by $t \approx 280$ as the easy modules finish, snaps back to 8 at $t = 320.3$ when the barrier, the report broadcast and the blame scatter release every rank within 0.1 s of each other, decays again, peaks at 7 around $t = 490$ as the repair round starts, and then falls to 1 from $t \approx 690$ to the end while rank 6 finishes alone. The 24.4 % serial fraction is those two tails. Achieved parallelism is 3.81 against a world size of 8, so the run bought eight agents and used just under four.

The 88.3 % coordination share needs reading rather than quoting, because it does not mean the protocol is expensive. Of the 748.5 s of collective wall time, 680.5 s is the three barriers and 66.5 s is the `reviewback-r1` exchange, and a barrier's wall is by definition the time the first arrival waits for the last — it is a measurement of agent latency spread, not of fabric cost. The collectives that actually move data cost 1.40 s in total (0.026 + 0.430 + 0.428 s for the three broadcasts, 0.245 + 0.264 s for the two gathers, 0.004 + 0.002 s for the two scatters) and carried 120,565 tokens while doing it. The fabric is not the bottleneck in this run by three orders of magnitude; the model is.

Figure 4 shows one bright cell, $4 \to 0$ at 48,501 tokens, with $6 \to 4$ at 20,410 and $5 \to 4$ at 17,020 behind it. That is the binomial gather tree seen edge-on: rank 4 is the interior node that folds ranks 5, 6 and 7 and forwards the combined code bundle to the root for the acceptance run. The concentration is the algorithm working as specified, not a hot spot — the whole gather completes in 0.245 s — and it is invisible in cost because 128,769 of the 169,184 tokens sent were deferred under rendezvous rather than admitted into any rank's context.

The right-hand panel of Figure 3 is empty, and that is by construction rather than by fault: every collective in this harness is called with `admit=False`, so context occupancy is 0.0 % against a 120,000-token budget on all eight ranks and the context-pressure axis has nothing to draw. In the dashboard screenshot the same emptiness appears as a rank-health table with no amber anywhere.

Two things in the viewer are worth naming because they can be misread. The violet glyphs at the right of every lane are labelled "fault recovery", but this run had zero trouble events: the 16 violet marks are the 16 `ft.agree` votes, two per rank, which the palette classes with the fault-tolerance primitives. And the legend counts 64 work items where the log holds 32 `agent.call` events, because the viewer builds one occupancy span from each `broker.claim`/`broker.complete` pair and then draws the `agent.call` instant on top of it; there are 32 bars on screen, four per lane. Neither is a protocol fault, but a reader who trusts the legend will over-count both.

7 Interpretation

7.1 What is true by construction

The run completed with all eight ranks finalized, no failed ranks, no contract violations across 32 invocations, no retries, and `degraded` false, so nothing here is a story about the fabric breaking. The executors are `broker`×8 with 46 worker attachments, which means real agents served every call; this is evidence about agent behaviour, not about a surrogate. The 46 reattachments and the maximum incarnation of 8 are the broker model working as designed — a rank role, its mailbox and its context account outlive the process serving it — and their distribution is informative rather than alarming: reattachment counts of 7, 7, 7, 6, 5, 5, 5 and 4 for ranks 0 through 7 are perfectly rank-order anticorrelated with busy times of 156.7 through 761.5 s, which is what you expect if workers are recycled between calls and a long call holds its worker.

The absence of any `win.` event is also by construction: with `-no-shared-interfaces` the harness creates no window and substitutes a `PROCEED` barrier labelled `interfaces-skipped`, which is duly present at $t = 42.5$ with a 31.2 s wall. The interface phase still runs. Eight agents still wrote eight interface documents, costing 13,862 prompt and 9,959 completion tokens — 12.4 % of the run's 191,602 tokens — and those documents were written to no channel. They are not entirely wasted, because the harness feeds a rank its own published interface back into its implementation prompt as a commitment it must honour, but no other rank ever saw one.

7.2 What the two-by-two actually shows

The design point is the low corner of a two-by-two over *specification precision* and *interface channel*. Reading it required recovering two numbers, because the results recorded for the two precise-spec cells are wrong. Both `results/software/real-sw-p8-noshared-locks-review.json` and its `shared` sibling record `importable: false` with `n_total` 0, and their `import_error` field contains a mid-slice of the oracle's own JSON report full of `"passed": true`. That is the signature of the `out.rfind("{")` parse bug present at commit `49f2bba` and fixed in `1f7bd01` at 07:59:39; the two runs started at 07:47:06 and 07:55:25 and so executed the broken code, and their provenance stamps a later commit only because provenance is captured when the result is written. I recovered the true values by copying each run's `workspace/round2` tree to a scratch directory outside the repository and running the unmodified oracle (`experiments/minidb/acceptance.py`) against it. The file modification times of those trees fall inside their runs' trace windows, so they are the artefacts those runs produced.

	interface window	**no window**
precise spec	`real-sw-p8-full` 60/60 at round 1; 2,002 lines 161,989 tokens; $1.1444	`real-sw-p8-noshared` 59/60, never green; 2,696 lines 167,174 tokens; $1.3767
vague spec	`vague-sw-p8-vague-shared` 60/60 at round 1, stops; 2,195 lines 86,473 tokens; $0.6471	**this run** 55/60 then 60/60; 3,556 lines 191,602 tokens; $1.6427

Pass rates for the two precise-spec cells are replays of the oracle against the trees those runs left on disk, for the reason given above; the recorded results for both say `importable: false`. Line counts are the eight generated `minidb/*.py`

files of the final round. Token and cost figures are the runs' own totals.

The interaction is the finding, and it is visible in four independent measures. Under a precise specification, withholding the window costs 35 % more lines, 3 % more tokens, 20 % more dollars and one acceptance case. Under a vague specification it costs 62 % more lines, 122 % more tokens, 154 % more dollars and a whole extra round. That is the confound the vague-spec variant was built to remove, stated as a measurement: when the broadcast specification already pins every signature, it *is* a shared interface and the window has little left to contribute; when it does not, the window is the only channel and its absence is expensive. This run is the cell that makes the point.

The cost structure is sharper than the totals suggest, and it is the single result I would carry forward. Splitting both vague runs by phase from their `agent.call` events:

phase	`vague-shared`		this run	
	prompt	completion	prompt	completion
publish interfaces	13,862	9,946	13,862	9,959
implement, round 1	40,307	22,358	24,713	33,764
review, round 1	—	—	32,208	7,108
implement, round 2	—	—	31,830	38,158
total	54,169	32,304	102,613	88,989

The interface phase is identical in both arms to within 13 completion tokens, as it must be: the prompts are byte-identical and neither arm has read anything yet. The difference appears in round one, and it points the way you would hope. The windowed arm's implementation prompts carry 15,594 more input tokens, which is the published interface text it is reading, and it emits 11,406 *fewer* completion tokens, which is the code it did not have to write to work around not knowing. Through round one the two arms cost almost the same: 82,298 tokens here against 86,473 for the entire windowed run. The windowless arm's first attempt was in fact 4.8 % cheaper — and it scored 55 of 60 instead of 60 of 60. The 109,304 tokens of review and repair that followed are the whole of the difference. Reading an interface cost about 15.6 k tokens and saved about 109 k, a ratio near seven to one.

7.3 Did they converge, and how

They did, but by decoupling rather than by agreeing, and the trace records the decision explicitly. Counting the eight declared dependency edges that are not `errors` (which everything imports and which the specification names outright): `parser` → `tokens`, `parser` → `nodes`, `planner` → `nodes`, `engine` → `planner`, `engine` → `functions`, and `api` → `parser`, `planner`, `engine`. In round one, three of those eight appear as imports in the generated tree, all three of them `api`'s. In round two, five of eight do. In `vague-sw-p8-vague-shared`, all eight do. The three edges never honoured here are exactly the three that point at a module whose interface was withheld and whose job could be duplicated: `parser.py` contains its own lexer and its own AST classes, which it then exports, and `planner.py` builds its own resolved-expression IR. The consequence is that `nodes.py`, 378 lines written by rank 2, is imported by nothing in the final tree, and `tokens.py` is reached only through `api.py`, which calls `tokenize(sql)` on every query and then discards the result because `inspect.signature` tells it the parser wants a string.

The agents said so, in the `concerns` field of their own implementations. The `parser` owner: "No interface was published to me for `minidb/tokens.py` or `minidb/nodes.py`… Importing names from

those modules would mean inventing signatures for modules I do not own," and then, remarkably, "The node types are exported from `parser.py` so a dependent that likewise received no interface for `nodes.py` can still work against something concrete." That is a rank unilaterally appointing itself the interface publisher because the channel that was meant to do it was removed. The `engine` owner: "the plan is read structurally through one helper (`_pick`) that accepts either attributes or mapping keys and tolerates the obvious synonyms," and "Aggregates are supported in both plausible planner encodings" — it wrote two implementations and kept both. The `api` owner: "Verified locally against four sets of stub sibling modules." None of this is guessing at a neighbour's signature, which the prompt forbids; it is the strictly more expensive alternative of accommodating every signature the neighbour might have.

That behaviour has a measurable signature. Across the eight generated files, this run's final tree contains 10 `getattr` calls, 2 `inspect` uses and 77 `: Any` annotations, against 0, 0 and 28 for the windowed arm on the same specification and the same commit; `real-sw-p8-noshared` has 25 `getattr` calls against 5 for `real-sw-p8-full`. Normalised for size, `: Any` appears 2.17 times per hundred lines here and 1.28 times per hundred in the windowed arm. The clearest single artefact is `api.py`'s `_parser_wants_tokens`, which inspects the parser's first parameter name and annotation for the substrings `token`, `sql`, `str`, `text` and `source`, calls the parser with whichever argument it guesses, and falls back to the other on `TypeError`. That is interface negotiation implemented in the caller at run time because no negotiation channel existed at build time.

The cost is localised precisely where the mechanism predicts. The four dependency-free modules — `errors.py` (31 lines), `tokens.py` (222), `nodes.py` (302) and `functions.py` (248) — are *byte-identical* between this run and `vague-sw-p8-vague-shared`, verified by MD5. The divergence is entirely in the four modules with dependencies: `parser` 437 → 730 lines, `planner` 549 → 766, `engine` 342 → 804, `api` 64 → 206, comparing the windowed arm to this run at round one. The same pattern holds for the precise-spec pair, whose four leaf modules are likewise identical to each other. A single run per cell cannot establish an effect size, but this is not a difference of two noisy draws: half the artefact is bit-for-bit the same and the cost falls only on the half where the mechanism could possibly act.

7.4 Why 482 events and not 311, and what the extra activity was

The difference between this run's 482 events and the windowed arm's 311 is not more negotiation. It is a review round and a repair round that the windowed arm never needed. The windowed arm made 16 agent calls, one interface and one implementation per rank, scored 60 of 60 at first integration, and `agree` returned true, so the loop broke before review. This run made 32 calls across four phases and its round-one `agree` returned false at $t = 320.3$, which is what put the review and repair phases on the timeline at all. Sixteen `topo.graph_create` events — two graphs per rank, the review digraph and its transpose — and 16 `coll.neighbor_allgather` events are the review machinery that only exists in this arm.

The extra activity was productive rather than thrashing, and the trace lets one say so sharply. Round one failed five cases: `order_desc`, `order_nulls_last_desc`, `order_by_alias`, `join_group` and `empty_table_select`. Four of the five are one root cause. Round-one `parser.py` declares its ORDER BY item as `ascending: bool = True` while round-one `planner.py` probes the item for a field named `descending` or `desc`, finds neither, and falls through to `bool(None)`, so every DESC in the suite silently planned as ASC. The peer review named it exactly: the `engine` owner, reviewing `parser` and `planner`, wrote "ORDER BY direction polarity mismatch. `parser.OrderItem` carries

`ascending: bool = True`; planner publishes `OrderItem.descending`. A structural read of 'descending'/'desc' finds no such field, so DESC silently b[ecomes ASC]." The fifth failure was named twice, by the owners of `api` and `errors`, both pointing at `engine.py`'s `raise QueryError("query has no output columns")` firing on `SELECT * FROM e` over an empty table. Every one of the five acceptance failures was identified by name in the review round, before the repair round, by an agent that did not own the failing module and was reading only a code excerpt. The reviews carry no acceptance results — the review prompt contains source, not failures — so this is diagnosis from the code, not restatement of the oracle.

The repair is equally legible, and it shows the price of having no channel even when the outcome is right. In round two the parser flipped its field to `descending: bool = False` and added an `ascending` property for compatibility; independently, the planner grew a `_descending_of` helper that probes `descending`, then `desc`, then `ascending`, then `asc`, negating as required. Both sides fixed the same mismatch, in ignorance of each other, and the fix is compatible because each was defensive enough to accept the other's convention. That is convergence, but it is convergence paid for twice, and the 247 extra lines between round one and round two are largely what it cost.

7.5 The flagged findings

The two `warning` findings are the same finding twice and are a real defect, but in the trace emitter rather than in the protocol. `neighbor_allgather` reports zero messages for `review-r1` and `reviewback-r1` while the fabric logged 16 each. The arithmetic is exact: of 81 `msg.send` events, 49 carry the internal tags of the two broadcasts, two gathers and two scatters (7 each), and the remaining 32 carry `_ampi:nbr_ag:0:8` and `_ampi:nbr_ag:0:9`, 16 apiece. The neighbourhood collectives emitted in-degree and out-degree but no `messages_sent`, so every trace read zero traffic for them. Commit `ee21d79` fixes precisely this, on 2026-09-01, and this run is from 2026-08-30, so the trace predates the fix and cannot be re-emitted. What should be said in its favour is that the cross-check earned its keep: the check compares the collective's self-report against `msg.send` events carrying its tag, and it caught an omission that the collective's own accounting could not. The cost of the gap is visible in Figure 5, which plots 10 of the 12 collectives and silently omits the two that disagree, and in the dashboard's collectives panel, where `neighbor_allgather` shows 16 invocations and 0 messages next to `bcast`'s 21.

The 88.3 % coordination note is expected for this configuration and is not a defect, for the reason given in the timeline section: 680.5 of 748.5 collective seconds are barriers whose wall is agent latency spread, and the data-moving collectives cost 1.40 s. The 1.88× imbalance note is likewise expected and structural, being the difference between writing `errors.py` and writing `engine.py`. The 46 reattachments are the broker model working. The cost-model agreement — 10 of 10 checkable collectives sending exactly the predicted message count — is unremarkable and should be: it is the control that tells you the traffic anomaly above is in the reporting and not in the algorithms.

8 Threats to this reading

The withholding condition was misstated to the agents, and the misstatement is perfectly confounded with the treatment. This is the most serious threat and it is specific. Every one of the 16 implementation prompts in this run, in both rounds, contains the line "PUBLISHED INTERFACES OF YOUR DEPENDENCIES: (this module has no dependencies)" — including the prompts for `parser`, `planner`, `engine` and `api`, which have three, one, two and three dependencies

respectively. That is false, and the harness author later said so: commit `f121647`, at 09:41 on the same day, replaces it with an explicit "NOT AVAILABLE… their authors' published interfaces have not been shared with you in this configuration" branch, on the stated grounds that "a condition that withholds information must say so; it must not misstate the structure." This run finished at 09:14, 27 minutes before that fix. In the windowed arm, by contrast, only the two genuinely dependency-free modules saw that line and every dependent module received real interface text, so the misstatement is present in exactly the arm under test and absent from its control. Strictly, this run measures "no window, and each rank falsely told it has no dependencies", and some unknown part of the decoupling I have attributed to the missing channel may be attributable to the false premise instead. Two things bound the damage. The vague module-layout table survives in the specification and names each module's permitted imports, so the structure was recoverable; and at least two ranks recovered it and said so — the `parser` owner wrote "even though the module layout lets `parser.py` import them", and the `api` owner wrote "the prompt states this module has no dependencies, yet `api.py` is the module whose entire job is to tie errors/tokens/parser/planner/engine together." The agents caught the harness. But `engine` and `planner` did not object in the same terms, and I cannot rule out that the false premise pushed them further towards structural probing than a truthful withholding notice would have.

Two of the four cells are compromised as comparison points. Both `real-sw` runs executed the broken `parse_report` and were therefore told, at both integrations, that their build did not import. The harness treats an import failure as everyone's problem and appends an `IMPORT` blame item to all eight ranks, so those two runs spent their entire round two repairing a phantom, and their `agreed_green` is false for a reason unrelated to their code. Their token and dollar totals therefore include a wasted round that the vague-shared arm did not pay — symmetrically across the precise pair, which is why I still quote their ratio, but it means the top row of the two-by-two is not directly commensurable with the bottom row. The comparison I lean on is the vague pair alone: same commit (`90ca469`), same specification, runs seven minutes apart, differing only in `shared_interfaces`.

The two vague arms are not fully independent draws. Nine prompts are byte-identical across the two runs and seven of the nine returned byte-identical results, with latencies differing by up to 4.4× (`iface:nodes`: 28.6 s here, 127.1 s there), so the executor re-ran them rather than serving a cache, but its sampling is close to deterministic. That cuts both ways. It strengthens the localisation claim, because the identical leaf modules are a genuine control rather than a coincidence, and it weakens any claim that the two runs bracket the run-to-run variance of this population, because on identical input this population barely varies. Nothing here estimates the variance of a re-run with a different seed, and a single run per cell cannot.

Wall-clock ratios are the least trustworthy of the four measures. The 2.11× wall-time ratio between the arms is contaminated by broker and model load: the eight interface calls, whose prompts are byte-identical in both runs, summed to 221.6 s of agent latency here and 545.7 s in the windowed arm. If identical work can differ by 2.5× in service time between two runs eight minutes apart, no wall-time difference of that magnitude should be attributed to the protocol. The token counts, which are properties of what was generated rather than of when, are the measure to trust, and the line counts and import graphs more so still.

Pass rate is a coarse and slightly generous oracle here. Sixty cases is not a large suite, 37 of them are attributed to `engine`, and the four DESC failures of round one are one bug counted four times, so "55 of 60" overstates how close round one was in one direction and understates the severity of a single interface mismatch in the other. More importantly, the suite tests behaviour and

not structure: a tree in which `parser.py` reimplements the lexer and `nodes.py` is dead code passes 60 of 60 exactly as a well-composed one does. The whole cost of the missing channel in this run is invisible to the metric the harness optimises, and is visible only in the artefact. Any conclusion of the form "the ablation was free because both arms reached 60/60" would be wrong for that reason, and the import graph is the evidence that makes it wrong.

Some of my numbers are not in `metrics.json`. The pass rates for the two precise-spec cells are oracle replays I ran, as described above; the line counts, MD5 comparisons, import-edge counts and `getattr`/`inspect`/`Any` tallies are computed from the generated trees in `runs/*/workspace/`; the per-phase token table is aggregated from the `agent.call` events of each run's log; and the quoted concerns and review findings are from the run's own `spool/call-*.result` files. Nothing in the repository was modified to obtain them. They should be read as reproducible measurements on the recorded artefacts rather than as harness outputs, and if the artefacts on disk were ever overwritten by a later run the line counts would be wrong — I checked that the modification times of every tree I measured fall inside its own run's trace window, but that check is weaker than a content hash recorded at the time.